

Lab 01: Prompt a REST API Contract

Maps to: A1, A2

Convert Northstar Commerce requirements into a versioned OpenAPI-first route and data contract.

AI Vibe Coding Assets

- ai_vibe.py - OpenAI Python SDK runner
- mock-data.json - synthetic request and test context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - first-run and refinement prompts
- working-files/generated/ - isolated AI output for review

First-Run Prompt

Create a behaviour-first OpenAPI 3.1 contract for Northstar Commerce using FastAPI and SQLite. Define Product, Customer, Order and OrderItem schemas; list collection/member routes, request and response JSON, 200/201/204/401/404/409/422 outcomes, and at least six acceptance tests. Do not write implementation code yet. Return only api-contract.md, openapi-draft.yaml and machine-readable test cases.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
pytest -q
```

Detailed procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 7: Install the Microsoft Python, Pylance, REST Client, SQLite Viewer and Red Hat YAML extensions in VS Code.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 8: Open the scenario and identify Product, Customer and Order resources plus their relationships.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 9: Create api-contract.md with collection and member routes under /api/v1.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 10: Draft OpenAPI 3.1 paths, schemas, responses and API-key security in openapi-draft.yaml.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 11: Ask the assistant to critique ambiguous identifiers, money fields, stock rules and order state transitions.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 12: Validate the YAML and save the accepted contract decisions with the prompt evidence.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Acceptance criteria

The contract covers products, customers and orders, names 200/201/204/401/404/409/422 outcomes and validates as OpenAPI 3.1.